

Motivation

As we discussed in the ALU assignment, we can design the ALU to perform the required operations such as addition, subtraction, multiplication, and division using operators like $+$, $-$, $*$, and $/$.

In this case, we let the synthesis tool decide how to implement these operators in order to meet our design constraints.

But what happens when we need to achieve a higher operating frequency?

Using the operators directly does not always give us the best architecture for our specific timing, area, or power requirements. Therefore, we can use specific arithmetic architectures that are designed to provide different trade-offs between area, power, and frequency.

For example, there are several architectures such as:

- Ripple Carry Adder (RCA)
- Carry Look-Ahead Adder (CLA)
- Kogge-Stone Adder
- Brent-Kung Adder

These architectures are implemented at the gate level, and each one provides a different trade-off between speed, area, and power.

The same idea applies to multiplication. There are several multiplier architectures, such as:

- Booth Multiplier
- Wallace Tree Multiplier
- Baugh-Wooley Multiplier
- Shift-and-Add Multiplier

Each architecture has its own advantages depending on the required design constraints.

For simplicity, in this assignment, we will focus on the Shift-and-Add Multiplier and study how its architecture affects the design performance.

Shift-and-Add Multiplier

Shift-and-Add multiplier is a simple way to perform multiplication using shifting and addition instead of directly using the * operator.

The idea is to check each bit of the multiplier. If the bit is 1, we add a shifted version of the multiplicand to the result. The amount of shifting depends on the bit position.

For example:

$$A \times B = (A \times b_0) + ((A \times b_1) \ll 1) + \dots + ((A \times b_7) \ll 7)$$

So, the multiplication is basically converted into a number of shift and add operations.

The main advantage is that it is simple and area efficient, but the multiple additions can increase the critical path, which limits the maximum frequency compared with more advanced multiplier architectures.

8-bit Example: 31 × 42

Represent both numbers using 8 bits:

A = 31 = 00011111₂

B = 42 = 00101010₂

Partial Product Generation:

PP [0] = A & B [0] = 00000000₂

PP [1] = A & B [1] = 00011111₂

PP [2] = A & B [2] = 00000000₂

PP [3] = A & B [3] = 00011111₂

PP [4] = A & B [4] = 00000000₂

PP [5] = A & B [5] = 00011111₂

PP [6] = A & B [6] = 00000000₂

PP [7] = A & B [7] = 00000000₂

Align, Shift and Accumulate:

```
00000000 000000002 (PP [0] << 0)
+ 00000000 001111102 (PP [1] << 1)
+ 00000000 000000002 (PP [2] << 2)
+ 00000000 111110002 (PP [3] << 3)
+ 00000000 000000002 (PP [4] << 4)
+ 00000011 111000002 (PP [5] << 5)
+ 00000000 000000002 (PP [6] << 6)
+ 00000000 000000002 (PP [7] << 7)
```

00000101 00010110₂

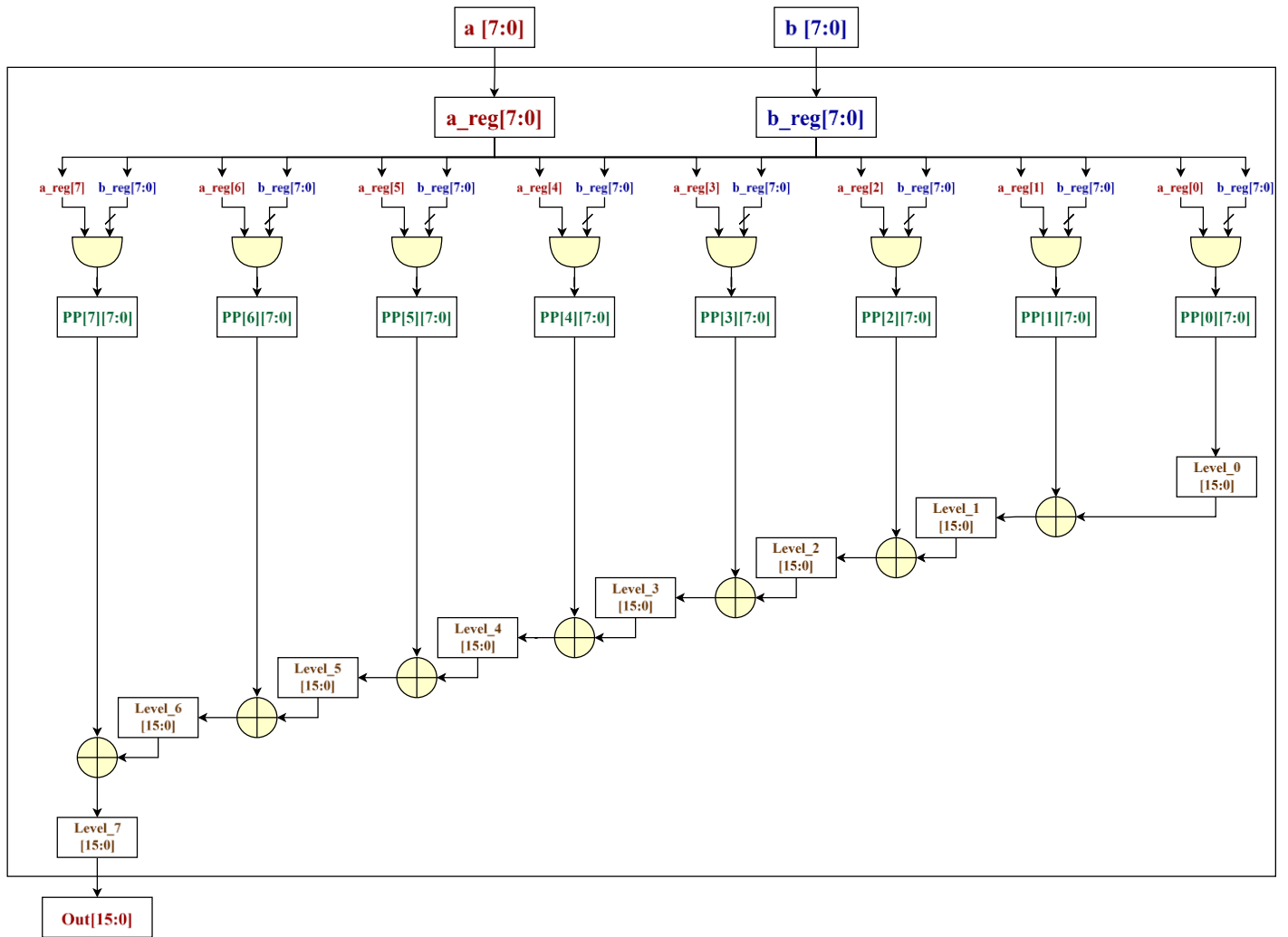


Figure 1: Block diagram for the multiplier

Design Overview

The `alu8_top` module is an 8-bit Arithmetic Logic Unit (ALU) that supports addition and multiplication. The design follows a 2-stage pipelined architecture, to stabilize signals and handle clock boundary timing

(Input Flip-Flops → Combinational Logic → Output Flip-Flops)

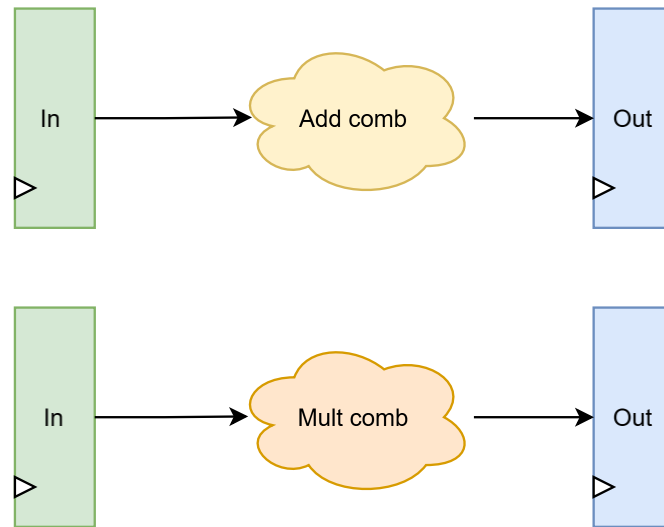


Figure 2: Design overview

- **alu8_top (Top Wrapper):** Routes inputs a and b to both sub-modules and multiplexes the final output based on mode (0 for Addition, 1 for Multiplication).
- **Adder Unit:** An 8-bit Adder that latches input operands on the clock edge, calculates the sum and carry-out combinationaly, and registers the final 9-bit result on the next clock edge.
- **Mult_unit:** Generates partial products using an 8×8 AND array and accumulates them through combinational shift-add logic. The final 16-bit product is registered on the output stage.

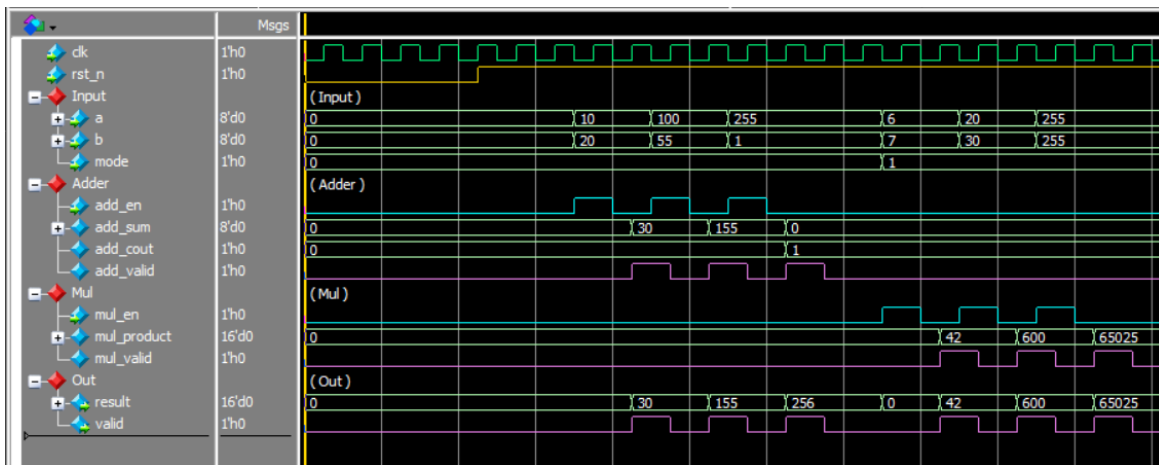


Figure 3: Design waveform

```

# =====
# ALU8_TOP TESTBENCH
# =====
#
# ----- ADD TESTS -----
# [ 85000] PASS | ADD | a=10 b=20 | result=30
# [105000] PASS | ADD | a=100 b=55 | result=155
# [125000] PASS | ADD | a=255 b=1 | result=256
#
# ----- MUL TESTS -----
# [165000] PASS | MUL | a=6 b=7 | result=42
# [185000] PASS | MUL | a=20 b=30 | result=600
# [205000] PASS | MUL | a=255 b=255 | result=65025
#
# =====
# SUMMARY
# -----
# Total tests : 6
# Passed      : 6
# Failed      : 0
# RESULT: ALL TESTS PASSED
# =====

```

Figure 4: Simple testbench results

- The waveform demonstrates two-cycle pipeline latency from when inputs are applied to when valid goes high and data is ready
- The testbench (tb.v) tests simple test cases to ensure that the design functions correctly.

Requirements:

1. Run synthesis on the given design starting with a 10 ns clock period, then gradually decrease the clock period to find the maximum frequency achievable without any timing violations. No constraints should be changed except the clock period.
2. After synthesis, identify the critical path and determine which design unit contributes the most to the critical delay.
3. Modify the critical unit by adding a pipeline register to the critical path. This increases the maximum latency of that unit from 2 cycles to 3 cycles.
4. Run synthesis again with the modified design and repeat the clock-period sweep until finding the new maximum frequency without timing violations.